

STANDARD CHANNEL MANAGER (Availability, Rates and Reservations) – Testing

Notes:

We are using **Postman** (free desktop tool) to show you how you can call our APIs

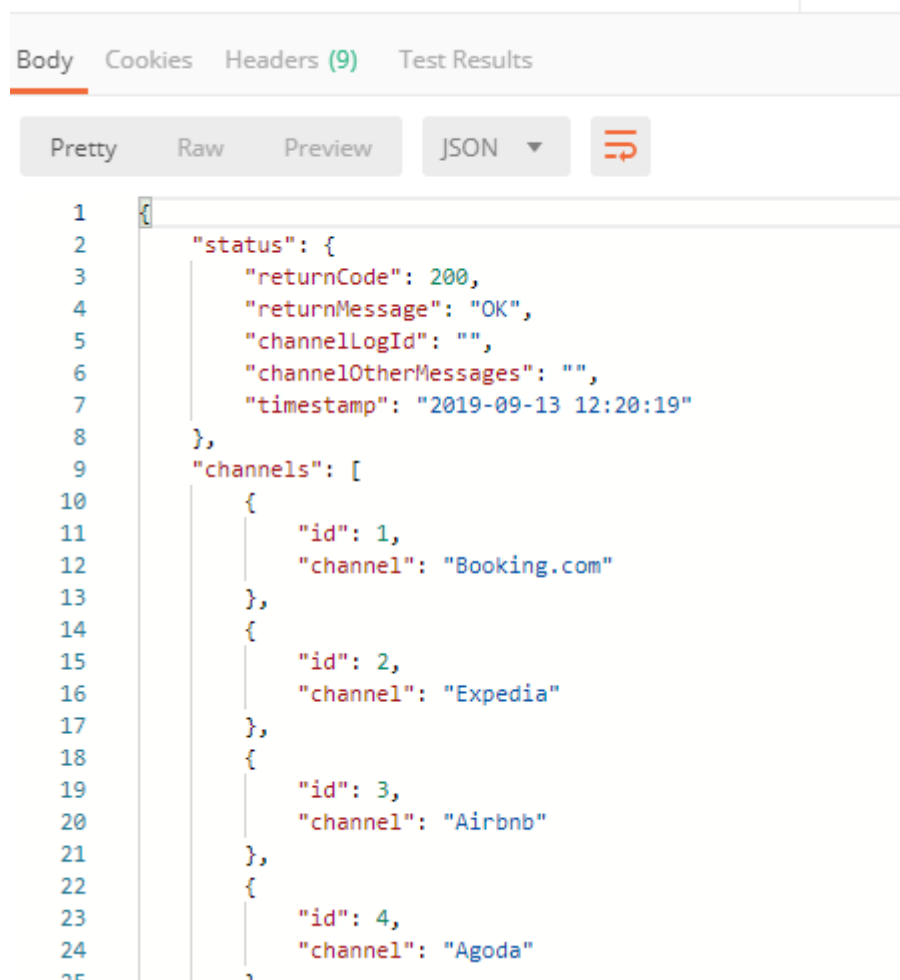
Authorization type: Basic Auth with the provided Username and Password

1. Test your connection

API: GET /channels

Use GET channels to test if your API test keys are correct.

Result:



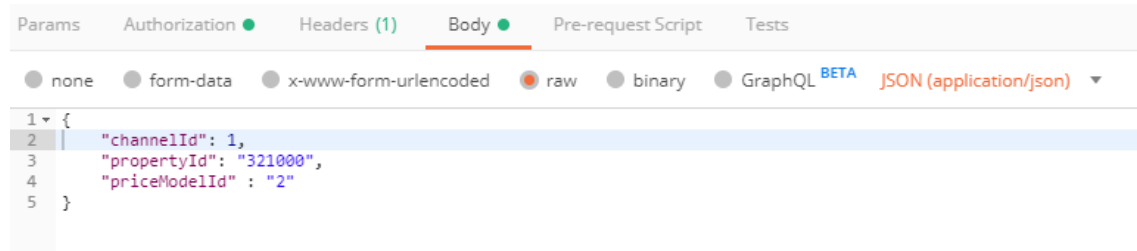
```
1 {
2   "status": {
3     "returnCode": 200,
4     "returnMessage": "OK",
5     "channelLogId": "",
6     "channelOtherMessages": "",
7     "timestamp": "2019-09-13 12:20:19"
8   },
9   "channels": [
10    {
11      "id": 1,
12      "channel": "Booking.com"
13    },
14    {
15      "id": 2,
16      "channel": "Expedia"
17    },
18    {
19      "id": 3,
20      "channel": "Airbnb"
21    },
22    {
23      "id": 4,
24      "channel": "Agoda"
25    }
26  ]
27 }
```

2. Activate your property in Zodomus

API: **POST /property-activation**

Activate a test property with Zodomus.

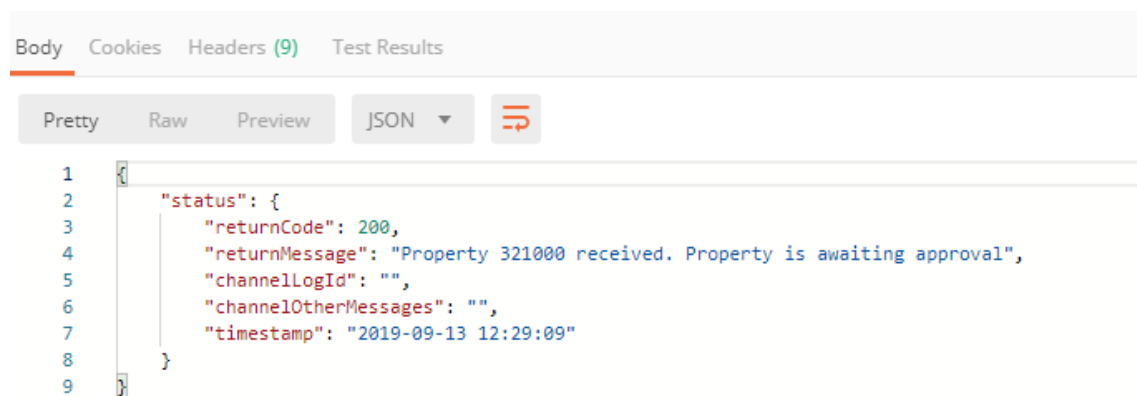
Json body:



The screenshot shows a REST client interface with tabs for Params, Authorization, Headers (1), Body, Pre-request Script, and Tests. The 'Body' tab is selected, and the 'JSON (application/json)' format is chosen. The request body is a JSON object with the following structure:

```
1 {  
2   "channelId": 1,  
3   "propertyId": "321000",  
4   "priceModelId": "2"  
5 }
```

Result:



The screenshot shows the same REST client interface with the 'Test Results' tab selected. The response body is displayed in a 'Pretty' JSON format. The response is a JSON object with a 'status' field containing a 'returnCode' of 200, a 'returnMessage' about property approval, and a 'timestamp'.

```
1 {  
2   "status": {  
3     "returnCode": 200,  
4     "returnMessage": "Property 321000 received. Property is awaiting approval",  
5     "channelLogId": "",  
6     "channelOtherMessages": "",  
7     "timestamp": "2019-09-13 12:29:09"  
8   }  
9 }
```

3. GET Room and Rates

API: **GET /room-rates**

Get the property rooms and rates for future mapping

GET parameters:

```
room-rates?channelId=1&propertyId=321000
```

Params	Authorization	Headers	Body	Pre-request Script	Tests
Query Params					
	KEY	VALUE			
☒	channelId	1			
☒	propertyId	321000			

Result:

Body	Cookies	Headers (9)	Test Results
PrettyRawPreviewJSON <pre>1 { 2 "status": { 3 "returnCode": 200, 4 "returnMessage": "OK", 5 "channelLogId": "", 6 "channelOtherMessages": "", 7 "timestamp": "2019-09-13 12:34:41" 8 }, 9 "rooms": [10 { 11 "id": "32100001", 12 "name": "Single room", 13 "rates": [14 { 15 "id": "321000991", 16 "name": "Non refundable", 17 "active": "1", 18 "maxPersons": "1", 19 "policy": "", 20 "policyId": "" 21 }, 22 { 23 "id": "321000992", 24 "name": "Standard rate", 25 "active": "1", 26 "maxPersons": "1", 27 "policy": "", 28 "policyId": "" 29 } 30] 31 } 32] 33 }</pre>			

4. GET Availability

API: GET /availability

Get the property rooms availabilities

GET parameters:

```
/availability?channelId=1&propertyId=321000&dateFrom=2019-09-01&dateTo=2019-09-03
```

Params	Authorization	Headers	Body	Pre-request Script	Tests
Query Params					
	KEY	VALUE			
☒	channelId	1			
☒	propertyId	321000			
☒	dateFrom	2019-09-01			
☒	dateTo	2019-09-03			

Result:

Body	Cookies	Headers (9)	Test Results
Pretty Raw Preview JSON			
<pre>1 { 2 "status": { 3 "returnCode": 200, 4 "returnMessage": "OK", 5 "channelLogId": "UmFuZG9tSVYkc2R1Iyh9Yf0c+r9MyekBpHKML4ATiZlmdGaCkvfZUIqNsLayoq+SV8TR78MzKiyNtnSLUdIjkVv31vFUwB/7", 6 "channelOtherMessages": "", 7 "timestamp": "2019-09-13 12:37:07" 8 }, 9 "rooms": [10 { 11 "id": "32100001", 12 "dates": [13 { 14 "date": "2019-09-14", 15 "availability": "1", 16 "booked": "0", 17 "cancelled": "0", 18 "rates": [19 { 20 "booked": "0", 21 "cancelled": "0", 22 "closed": "0", 23 "closedOnArrival": "0", 24 "closedOnDepart": "0", 25 "exactStayArrival": "0", 26 "lengthOfStay": "0",</pre>			

4. POST Rooms activation

API: **POST /rooms-activation**

Map the RoomId and RateIds

Body Json:

```
none form-data x-www-form-urlencoded raw binary
1 {
2   "channelId": 1,
3   "propertyId": "321000",
4   "rooms": [{
5     "roomId": "32100001",
6     "roomName": "Single room",
7     "quantity": 1,
8     "status": 1,
9     "rates": [
10      "321000991",
11      "321000992",
12      "321000993"
13    ]
14   },
15   {
16     "roomId": "32100002",
17     "roomName": "Double room",
18     "quantity": 2,
19     "status": 1,
20     "rates": [
21      "321000991",
22      "321000992",
23      "321000993"
24    ]
25   },
26   {
27     "roomId": "32100003",
28     "roomName": "Suite",
29     "quantity": 2,
30     "status": 1,
31     "rates": [
32      "321000991",
33      "321000992",
34      "321000993"
35    ]
36   }
37 ]
38 }
```

Result: **Body** Cookies Headers (9) Test Results

Pretty Raw Preview JSON

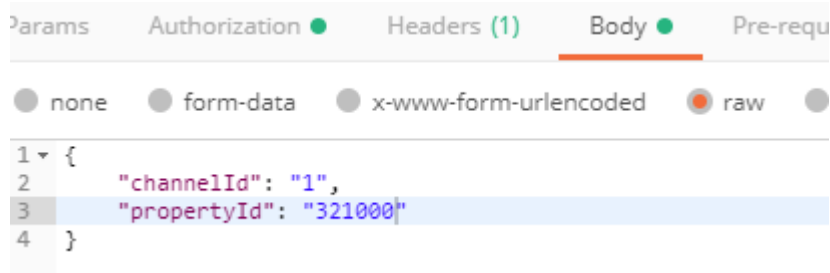
```
1 {
2   "status": {
3     "returnCode": 200,
4     "returnMessage": "Number of rooms activated: 3",
5     "channelLogId": "",
6     "channelOtherMessages": "",
7     "timestamp": "2019-09-13 12:42:38"
8   }
9 }
```

5. POST Property check

API: POST /property-check

Used to verify if the property status is active and rooms and rates are mapped.

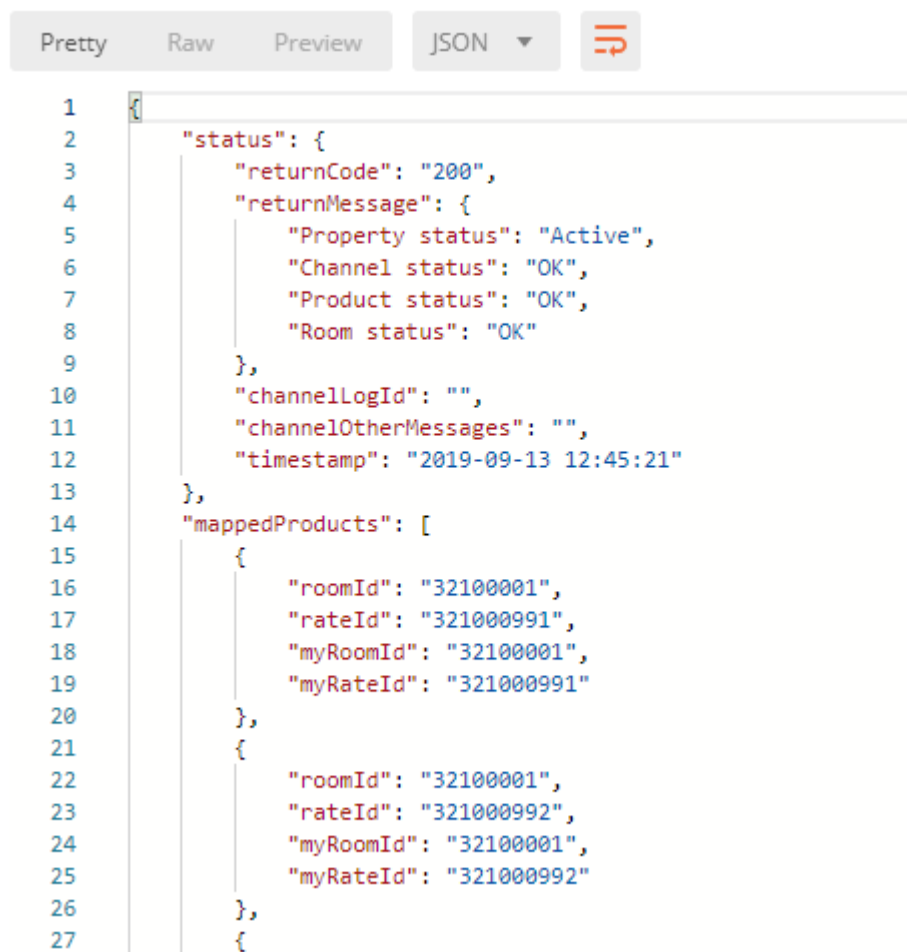
Body Json:



The screenshot shows a REST client interface with tabs for Params, Authorization, Headers (1), Body, and Pre-reqs. The Body tab is selected, and the payload is set to 'raw'. The JSON body is as follows:

```
1 {  
2   "channelId": "1",  
3   "propertyId": "321000"  
4 }
```

Response:



The screenshot shows a REST client interface with tabs for Pretty, Raw, and Preview. The Pretty tab is selected, and the response is displayed in JSON format. The response is as follows:

```
1 {  
2   "status": {  
3     "returnCode": "200",  
4     "returnMessage": {  
5       "Property status": "Active",  
6       "Channel status": "OK",  
7       "Product status": "OK",  
8       "Room status": "OK"  
9     },  
10    "channelLogId": "",  
11    "channelOtherMessages": "",  
12    "timestamp": "2019-09-13 12:45:21"  
13  },  
14  "mappedProducts": [  
15    {  
16      "roomId": "32100001",  
17      "rateId": "321000991",  
18      "myRoomId": "32100001",  
19      "myRateId": "321000991"  
20    },  
21    {  
22      "roomId": "32100001",  
23      "rateId": "321000992",  
24      "myRoomId": "32100001",  
25      "myRateId": "321000992"  
26    },  
27  ]  
}
```

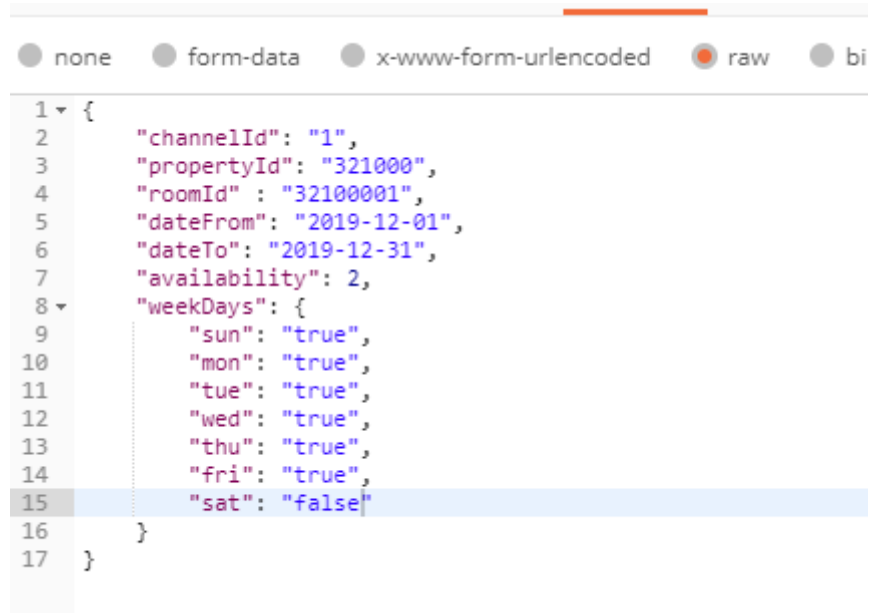
Since the result is Ok, we can now connect to the property to send Rates, Availabilities and receive Reservations.

5. POST Availabilities

API: POST /availability

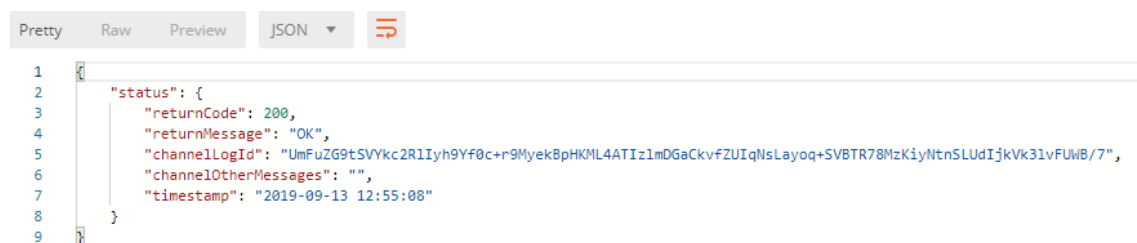
Used to set availability for a room in a specific channel

Body Json:



```
1 {
2   "channelId": "1",
3   "propertyId": "321000",
4   "roomId": "32100001",
5   "dateFrom": "2019-12-01",
6   "dateTo": "2019-12-31",
7   "availability": 2,
8   "weekDays": {
9     "sun": "true",
10    "mon": "true",
11    "tue": "true",
12    "wed": "true",
13    "thu": "true",
14    "fri": "true",
15    "sat": "false"
16  }
17 }
```

Result:



```
1 {
2   "status": {
3     "returnCode": 200,
4     "returnMessage": "OK",
5     "channelLogId": "UmFuZG9tSVYkc2R1Iyh9Yf0c+r9MyekBpHKML4ATIzlmDgaCkvfZUIqNsLayoq+SVBTR78MzKiyNtnSLudIjkVv31vFUwB/7",
6     "channelOtherMessages": "",
7     "timestamp": "2019-09-13 12:55:08"
8   }
9 }
```

Notes:

The availabilities are always set for each room.

The availability "2" is the number of available rooms for reservations on this channel

Setting availability to "0" will close the room (the room can also be closed with the closed status)

6. POST Rates

API: **POST /rates**

Used to send rates and restrictions for a room / rate

Body Json:

```
{
  "channelId": "1",
  "propertyId": "321000",
  "roomId": "32100001",
  "dateFrom": "2019-12-01",
  "dateTo": "2019-12-31",
  "currencyCode": "EUR",
  "rateId": "321000991",
  "weekDays": {
    "sun": "true",
    "mon": "true",
    "tue": "true",
    "wed": "true",
    "thu": "true",
    "fri": "true",
    "sat": "true"
  },
  "prices": {
    "price": "145"
  },
  "closed": "0"
}
```

Result:

```
Pretty Raw Preview JSON
1 {
2   "status": {
3     "returnCode": 200,
4     "returnMessage": "OK",
5     "channelLogId": "UmFuZG9tSVYkc2R1Iyh9Yf0c+r9MyyekBpHKML4ATIzlmDGaCkvfZUIqNsLayoq+SVBTR78MzKiyNtnSLUdIjkVvk3lvFUwB/7",
6     "channelOtherMessages": "",
7     "timestamp": "2019-09-13 12:55:38"
8   }
9 }
```

Notes:

Here you can set full room prices and restrictions (for instance : "closed" status)

The rates as restrictions are always applied to a single room / rate